Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry

Bakalárska práca

2026
Matúš Duchyňa

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry

Bakalárska práca

Študijný program:	Informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Katedra informatiky
Školiteľ:	doc. Mgr. Tomáš Plachetka, Dr.

Bratislava, 2026
Matúš Duchyňa

92123197

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Matúš Duchyňa
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry
Compiler of Datalog with function symbols to relational algebra

Anotácia: Definovať rozšírenie Datalogu o funkčné symboly. Skonštruovať vhodnú formálnu gramatiku v syntaxi ANTLR. Implementovať kompilátor do relačnej algebry. Integrovať s kompilátorom relačnej algebry. Overiť funkčnosť na vhodných vstupoch.

Vedúci: doc. Mgr. Tomáš Plachetka, Dr.
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.

Spôsob sprístupnenia elektronickej verzie práce:
archív

Dátum zadania: 21.10.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce

Čestné vyhlásenie: Čestne vyhlasujem, že celú bakalársku prácu na tému „Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry“, vrátane všetkých jej príloh a obrázkov, som vypracoval/vypracovala samostatne, a to s použitím literatúry uvedenej v priloženom zozname.

Poďakovanie: TODO

Abstrakt

Slovenský abstrakt v rozsahu 100-500 slov, jeden odstavec. Abstrakt stručne sumarizuje výsledky práce. Mal by byť pochopiteľný pre bežného informatika. Nemal by teda využívať skratky, termíny alebo označenie zavedené v práci, okrem tých, ktoré sú všeobecne známe.

Kľúčové slová: datalog, relačná algebra, kompilátor, databázový systém

Abstract

Abstract in the English language (translation of the abstract in the Slovak language).

Keywords:

Obsah

Úvod **1**

1 Datalog **3**

1.1 Definície 3

1.2 Rozšírenie datalogu o funkčné symboly 6

1.3 Syntax rozšíreného datalogu 7

1.4 Sémantika datalogu 9

2 Relačná algebra **13**

2.1 Definície 13

2.1.1 Učebnicová relačná algebra 13

2.1.2 Naša relačná algebra 17

2.2 Inklúzia medzi učebnicovou a našou relačnou algebrou 19

3 Kompilátor **23**

3.1 ANTLR 23

3.2 Gramatika datalogu pre ANTLR 23

3.3 Algoritmus prekladu 23

Záver **23**

Literatúra **27**

Zoznam obrázkov

Zoznam tabuliek

Úvod

Klasické moderné databázové systémy poskytujú používateľom na vytváranie dotazov zväčša deklaratívne jazyky, ako napríklad SQL. Výhodou deklaratívneho jazyka je to, že používateľ sa nemusí zaoberať samotným algoritmom, ktorý vypočíta výsledok dotazu. Súčasťou klasických databázových modelov je tzv. *optimalizér*. Tento komponent za pomoci rôznych metadát o databáze vytvorí zo SQL dotazu *vykonávací plán*, t.j. algoritmus výpočtu dotazu. Avšak optimalizér nie vždy vytvorí *optimálny* vykonávací plán. Prinútiť databázový systém používať konkrétny vykonávací plán nie je väčšinou jednoduché. Preto existujú experti, ktorí sa zaoberajú výlučne touto problematikou. Klasické postupy optimalizácie zahŕňajú používanie *hintov*, prepisovanie dotazu na iný ekvivalentný dotaz alebo explicitné nariadenie použitia konkrétneho plánu. Vykonávacie plány však väčšinou používajú veľké množstvo komplikovaných operácií s množstvom parametrov a bez kvalitnej dokumentácie. Kvôli tomu je písanie konkrétneho plánu nepraktické a neefektívne.

Náš databázový systém[1] používa ako dotazovací jazyk relačnú algebru. Konkrétne ide o relačnú algebru s piatimi operátormi a podporou funkčných symbolov. Viac o nej je uvedené v kapitole 2. Relačná algebra nám priamo umožňuje špecifikovať algoritmus materializácie dotazu. Pri jej návrhu sme sa zamerali na jednoduchosť, čo sa však odrazilo na veľkosti dotazov. *Loader*[2] z relačnej algebry do programovacieho jazyka Java a implementáciu jednotlivých funkcií v jazyku Java vytvorili kolegovia Biriukova [1] a Magát [2].

Relačná algebra je síce silným nástrojom na určenie presného algoritmu výpočtu dotazu ale pre bežného používateľa je príliš nízkoúrovňová. Tvorba zložitejších dotazov vyžaduje manuálne skladanie operátorov čo zvyšuje nečitateľnosť kódu a riziko chyby. Tu vzniká priestor pre datalog, deklaratívny dotazovací jazyk ktorý je založený na matematickej logike. Datalog umožňuje efektívne definovať zložité dotazy pomocou pravidiel, ktoré sú často omnoho stručnejšie ako ekvivalentné zápisy v SQL alebo relačnej algebre.

Cieľom mojej bakalárskej práce je pridať datalog s funkčnými symbolmi ako ďalší

[1] Pod *naším* databázovým systémom myslím experimentálny systém, na ktorom pracuje alebo pracovali môj školiteľ a kolegovia pod jeho vedením

[2] Kompilátor

dotazovací jazyk do nášho systému. Konkrétne ide o vytvorenie kompilátora z datalogu do relačnej algebry tak, aby bol použiteľný aj samostatne, bez väzby na konkrétny databázový systém. Dôležité bude spracovanie funkčných symbolov, ktoré sa v štandardnom datalogu (resp. prologu) nevyskytujú. Kompilátor musí byť schopný overiť syntaktickú a sémantickú správnosť datalogového programu a následne vygenerovať čo najoptimálnejší ekvivalentný kód v relačnej algebre.

Podobným softvérom je aj *Soufflé* [4]. Soufflé kompilátor taktiež vykonáva sémantickú kontrolu, ale následne preloží kód datalogu do tzv. *Relation Algebra Machine*. Z *RAM* sa potom vygeneruje vysoko optimalizovaný a paralelizovaný C++ kód. Naše riešenie sa síce nezameriava na rýchlosť preloženého dotazu, ale na jeho jednoduchosť. Náš datalog aj relačná algebra obsahujú iba tie najdôležitejšie komponenty, ale zároveň zanechávajú dotazovaciu silu.

Zásadným rozdielom oproti nášmu riešeniu je, že Soufflé nepodporuje funkčné symboly v podobe, ktorú by sme potrebovali. Rovnako Soufflé nepodporuje ani priame využívanie vstavaných funkcií databázy (*built-in functions*), čo obmedzuje jeho schopnosť spolupracovať s existujúcimi databázovými systémmi. Náš prístup naopak počíta so vstavanou podporou funkcií priamo v relačnej algebre a databáze, čím umožňuje vykonávať komplexné transformácie dát už počas samotného procesu materializácie.

Okrem rozdielu v cieľovej platforme (C++ vs. Java) je dôležitá aj miera abstrakcie vygenerovanej relačnej algebry. Zatiaľ čo RAM je nízkoúrovňová inštrukčná sada navrhnutá pre konkrétny stroj, naša relačná algebra si zachováva matematickú podobu a je bližšia teoretickým definíciám. To umožňuje jednoduchšiu kontrolu správnosti prekladu a prípadnú manuálnu kontrolu vygenerovaného prekladu.

Práca je rozdelená na tri časti. V kapitole 1 bližšie popisujeme datalog ako jazyk logiky a definujeme jeho rozšírenie o funkčné symboly. Pri snahe osamostatniť náš kompilátor od nášho databázového systému sme identifikovali požiadavky na relačnú algebru. Všetky zmeny[3] a ich odôvodnenia sú popísané v kapitole 2. V záverečnej kapitole 3 popisujeme samotný kompilátor, jeho jednotlivé časti vrátane gramatiky nášho datalogu a algoritmu prekladu.

[3] Oproti relačnej algebre definovanej v [1]

Kapitola 1

Datalog

1.1 Definície

V nasledujúcej kapitole formálne definujeme Datalog vo forme, v akej s ním budeme vo zvyšku práce pracovať.

Poznámka 1. $\top$ označuje pravdu (hodnotu *true*). $\bot$ označuje nepravdu (hodnotu *false*). $\Cup$ označuje hodnotu *unknown*.

TODO 1. Vstavané funkcie a predikáty.

Definícia 1.1.1. **n-árna relácia** r je definovaná ako $r \subseteq D_1 \times D_2 \times \cdots \times D_n$. Jednotlivé prvky relácie r nazývame **tuples**, **záznamy** alebo **usporiadané n-tice**. Jednotlivým zložkám n-tíc hovoríme **atribúty** relácie. Množinám $D_1, D_2, \ldots, D_n$ sa hovorí **domény** (alebo **typy**) atribútov relácie r.

Definícia 1.1.2. **Jazyk logiky** pozostáva z

1. predikátových symbolov
2. symbolov pre premenné
3. symbolov pre konštanty (reprezentujú prvky z domén)
4. logických symbolov ($\wedge, \neg$)
5. symbolov všeobecných kvantifikátorov ($\forall$)
6. symbolov pre definície ($\Rightarrow$)
7. čiarok (`,`)
8. a zátvoriek (`(`, `)`).

Definícia 1.1.3. Do **formúl** patria:

1. *atomické formuly* $p(t_1, t_2, \ldots t_n)$, kde p je n-árny predikát a $t_1, t_2, \ldots t_n$ sú konštanty alebo premenné
2. ak A a B sú formuly, tak potom aj $A \wedge B$ a $\neg A$ sú formuly
3. nič iné nie je formula

Definícia 1.1.4. $A \Rightarrow B$ je **implikácia**, ak A je formula a B je atomická formula.

Definícia 1.1.5. $\forall X \iota$ sa nazýva **kvantifikovaná implikácia**, kde X je premenná a ι je implikácia. Ak je kvantifikovaná každá premenná v kvantifikovanej implikácii, nazývame ju **klauzula**.

Definícia 1.1.6. $K_1 \wedge K_2 \wedge \cdots \wedge K_n$ nazývame **teóriou**, kde $K_1, K_2, \ldots, K_n$ sú klauzuly.

Definícia 1.1.7. Ohodnotenie premenných je funkcia, ktorá priradí každej premennej konštantu. Ohodnotenie o, ktoré premennej X_i priradí konštantu k_i (pre $i \leq n$) označujeme $o(X_1|k_1, \ldots, X_n|k_n)$. Keď vo formule A resp. implikácii ι resp. teórii $K_1 \wedge \cdots \wedge K_n$ nahradíme všetky premenné podľa ohodnotenia o, výslednú formulu označujeme ako $A[o]$ resp. $\iota[o]$ resp. $K_1 \wedge \cdots \wedge K_n[o]$.

Definícia 1.1.8. Interpretácia jazyka je funkcia, ktorá každej atomickej formule priradí jednu z hodnôt $\top, \bot, \Cup$.

Definícia 1.1.9. Nech $\mathcal{I}$ je interpretácia jazyka, φ je formula a o je ohodnotenie premenných. Potom $\mathcal{I} \models_\top \varphi[o]$ resp. $\mathcal{I} \models_\bot \varphi[o]$ resp. $\mathcal{I} \models_\Cup \varphi[o]$ označuje, že formula $\varphi[o]$ je pravdivá resp. nepravdivá resp. *unknown* vzhľadom na interpretáciu jazyka $\mathcal{I}$.

Pravdivosť formuly definujeme nasledovne:

1. $\mathcal{I} \models_\top \neg\varphi[o]$ ak $\mathcal{I} \models_\bot \varphi[o]$.
   $\mathcal{I} \models_\bot \neg\varphi[o]$ ak $\mathcal{I} \models_\top \varphi[o]$.
   $\mathcal{I} \models_\Cup \neg\varphi[o]$ ak $\mathcal{I} \models_\Cup \varphi[o]$.
2. $\mathcal{I} \models_\top \varphi[o] \wedge \phi[o]$ ak $\mathcal{I} \models_\top \varphi[o]$ a zároveň $\mathcal{I} \models_\top \phi[o]$.
   $\mathcal{I} \models_\bot \varphi[o] \wedge \phi[o]$ ak $\mathcal{I} \models_\bot \varphi[o]$ alebo $\mathcal{I} \models_\bot \phi[o]$.
   $\mathcal{I} \models_\Cup \varphi[o] \wedge \phi[o]$ v ostatných prípadoch.

3. $\mathcal{I} \models_\top (\varphi \Rightarrow \phi)[o]$ ak platí
   (a) $\mathcal{I} \models_\top \varphi[o]$ a zároveň $\mathcal{I} \models_\top \phi[o]$,
   (b) $\mathcal{I} \models_\Cup \varphi[o]$ a zároveň $\mathcal{I} \models_\top \phi[o]$,
   (c) $\mathcal{I} \models_\Cup \varphi[o]$ a zároveň $\mathcal{I} \models_\Cup \phi[o]$,
   (d) $\mathcal{I} \models_\bot \varphi[o]$.

   $\mathcal{I} \models_\bot (\varphi \Rightarrow \phi)[o]$ ak $\mathcal{I} \models_\top \varphi[o]$ a zároveň $\mathcal{I} \models_\bot \phi[o]$.
   $\mathcal{I} \models_\Cup (\varphi \Rightarrow \phi)[o]$ ak $\mathcal{I} \models_\Cup \varphi[o]$ a zároveň $\mathcal{I} \models_\bot \phi[o]$.
4. Pre kvantifikovanú implikáciu $\forall X \iota$ platí:
   $\mathcal{I} \models_\top \forall X \iota$ ak pre *ľubovoľnú* konštantu c platí $\mathcal{I} \models_\top \iota[X|c]$,
   $\mathcal{I} \models_\bot \forall X \iota$ ak pre *niektorú* konštantu c platí $\mathcal{I} \models_\bot \iota[X|c]$,
   inak $\mathcal{I} \models_\Cup \forall X \iota$.

Definícia 1.1.10. Ak **n-árny predikát p_r je prislúchajúci** n-árnej relácii $r \subseteq D_1 \times D_2 \times \cdots \times D_n$, potom v každej interpretácii $\mathcal{I}$ jazyka, ktorá obsahuje p_r, platí $\mathcal{I} \models_\top p_r(c_1, c_2, \ldots, c_n)$ ak $(c_1, c_2, \ldots, c_n) \in r$, inak $\mathcal{I} \models_\bot p_r(c_1, c_2, \ldots, c_n)$.

Definícia 1.1.11. Interpretáciu $\mathcal{I}$ teórie $K_1 \wedge \cdots \wedge K_n$ nazývame **model**, ak pre všetky $i \in \{1, \ldots, n\}$ platí $\mathcal{I} \models_\top K_i$.

Definícia 1.1.12. Majme teóriu $K_1 \wedge \cdots \wedge K_n$ a atomickú formulu Q. Potom teóriu $K_1 \wedge \cdots \wedge K_n \wedge Q$ nazývame **teória s dotazom** Q.

Definícia 1.1.13. Majme teóriu s dotazom $K_1 \wedge \cdots \wedge K_n \wedge Q$, kde $X_1, \ldots, X_k$ sú premenné v dotaze Q. **Výsledkom dotazu** (vzhľadom na model $\mathcal{I}$ teórie $K_1 \wedge \cdots \wedge K_n$) nazývame množinu usporiadaných k-tic

$$\{(c_1, c_2, \ldots, c_k) \,|\, \mathcal{I} \models_\top Q[X_1|c_1, X_2|c_2, \ldots, X_k|c_k]\}.$$

Ak dotaz Q neobsahuje žiadnu premennú a platí $\mathcal{I} \models_\top Q$, výsledkom dotazu je $\top$, inak je výsledkom $\bot$.

Poznámka 2. Môžeme si všimnúť, že dotazom sa vieme pýtať iba otázky typu *platí* $p(K_1, \ldots, K_n)$*?*, kde $K_1, \ldots, K_n$ sú konštanty. Nevieme sa pýtať na opak (otázky typu *neplatí... ?*).

Týmto dokážeme odpovedať na niektoré dotazy pre teórie, ktoré nemajú dvojhodnotový model. Ako príklad môžeme zobrať teóriu

$$\mathcal{T} = (\neg p \Rightarrow q) \wedge (\neg q \Rightarrow p)\,.$$

Jediná validná interpretácia $\mathcal{I}_V$ je $\mathcal{I}_V \models_\Cup p$ a $\mathcal{I}_V \models_\Cup q$ (ak by sme zobrali ľubovoľnú inú interpretáciu $\mathcal{I}$, došli by sme k sporu $\mathcal{I} \models_\top p \wedge \mathcal{I} \not\models_\top p$ resp. $\mathcal{I} \models_\bot p \wedge \mathcal{I} \not\models_\bot p$). Ak by sme sa vedeli pýtať na záporné dotazy, dostali by sme pre teórie s dotazom $\mathcal{T} \wedge p$ a $\mathcal{T} \wedge \neg p$ výsledok dotazu $\bot$. S našim prístupom dostaneme pre dotaz „*platí p?*“ $\bot$.

Definícia 1.1.14. Klauzulu

$$\forall X_1 \forall X_2 \ldots \forall X_n (A_1 \wedge A_2 \wedge \cdots A_i \wedge \neg A_{i+1} \wedge \neg A_{i+2} \wedge \cdots \wedge \neg A_m) \Rightarrow B$$

nazývame **bezpečnou**, ak sa každá z premenných $X_1, X_2, \ldots, X_n$ vyskytuje v niektorej z pozitívnych formúl $A_1, A_2, \ldots, A_i$. Teóriu, ktorá obsahuje iba bezpečné klauzuly, nazývame **bezpečnou teóriou**.

Poznámka 3. Vo zvyšku práce budeme pracovať iba s bezpečnými klauzulami a bezpečnými teóriami. Dôvodom je nejednoznačnosť výsledkov dotazov pre teórie, ktoré obsahujú nebezpečné klauzuly.

Zoberme si teóriu s dotazom

$$\mathcal{T} = (\forall X \neg p(X) \Rightarrow q(X)) \wedge q(X)$$

s interpretáciou $\mathcal{I}$ takou, že $\forall i \in \mathbb{N} : \mathcal{I} \models_\top q(i)$ a pre všetky ostatné konštanty c platí $\mathcal{I} \models_\perp q(c)$. Čo je potom ale výsledok dotazu našej teórie s dotazom $\mathcal{T}$? Intuitívne by to malo byť všetko, čo nie je prirodzené číslo. Čo všetko to ale je? Rozprávame sa iba o číslach? Alebo tam máme zarátať aj množiny? Čo všetko dovolíme vložiť do množín? Nevieme to jednoznačne určiť, pretože nebezpečná klauzula nám umožňuje vytvoriť nekonečne veľa výsledkov dotazov, ktoré sa líšia v tom, čo všetko obsahujú. Bezpečné klauzuly nám zaručujú, že výsledok dotazu bude vždy jednoznačný (vzhľadom na interpretáciu).

Príkladom bezpečnej teórie s dotazom môže byť

$$\mathcal{T}' = (\forall X r(X) \wedge \neg p(X) \Rightarrow q(X)) \wedge p(X)$$

s interpretáciou $\mathcal{I}'$ takou, že $\forall i \in \mathbb{N} : \mathcal{I}' \models_\top p(i)$ a $\forall c \in \mathbb{R} : \mathcal{I}' \models_\top r(c)$. V tejto teórii sme obmedzili množinu konštánt, ktoré má zmysel dopĺňať do predikátu p. Výsledok dotazu potom vieme jednoznačne určiť. Konkrétne sa bude jednať o množinu reálnych čísel bez prirodzených čísel ($\mathbb{R} - \mathbb{N}$).

1.2 Rozšírenie datalogu o funkčné symboly

Doteraz sme dosádzali do predikátov iba konštanty a premenné. Rozšírime jazyk logiky o tzv. funkčné symboly. Funkčný symbol f (s aritou n) reprezentuje ľubovoľnú funkciu $f : D_1 \times D_2 \times \cdots \times D_n \to D$.

Funkčné symboly chceme zaviesť preto, že samotné konštanty a premenné nestačia na prirodzený zápis zloženejších hodnôt. Veľmi užitočná zložitejšia hodnota je napríklad zoznam. Zoznam môžeme reprezentovať pomocov dvoch funkčných symbolov: `nil`, ktorý reprezentuje prázdny zoznam, a `pair`, ktorý reprezentuje dvojicu, z ktorej vieme zostrojiť ľubovoľný zoznam. Zoznam obsahujúci prvky a, b a c sa potom bude

reprezentovať termom `pair(a, pair(b, pair(c, nil)))`. Ďalším užitočnym funkčným symbolo je tzv. nasledovník `s`. Ten jednoducho pre každé prirodzené číslo n vráti jeho nasledovníka $n + 1$. S týmito dvoma (troma ak rátame aj 0-árny funkčný symbol `nil`) funkčnými symbolmi dokonca vieme simulovať ľubovoľný turingov stroj, a teda aj ľubovoľný algoritmus. Čo oproti datalogu pridáva obrovskú výpočtovú silu.

Definícia 1.2.1. Za **term** považujeme:

- ľubovoľný 0-árny funkčný symbol (ktorý považujeme za *konštantu*, môžeme zapisovať aj bez zátvoriek),
- ľubovoľnú premennú (väčšinou budeme značiť veľkými písmenami latinskej abecedy),
- ak f je n-árny funkčný symbol a $t_1, t_2, \ldots, t_n$ sú termy, potom $f(t_1, t_2, \ldots, t_n)$ je term.

Dôležité je, že term pozostáva z konečného množstva symbolov.

Rozširime definíciu jazyka logiky (1.1.2) o symboly označujúce funkčné symboly. Do predikátov (1.1.3 1.) potom nedosadzujeme premenné a konštanty, ale termy.

Definícia 1.2.2. Ground term je ľubovoľný term, v ktorom sa nevyskytujú premenné.

V definícii pre ohodnotenie premenných (1.1.7) sa daná funkcia predefinuje na funkciu, ktorá priradí každej premennej nie konštantu, ale *ground term*. Taktiež v definícii interpretácie jazyka (1.1.9) prepíšeme 4. časť na:

> Pre kvantifikovanú implikáciu $\forall X \iota$ platí:
> $\mathcal{I} \models_\top \forall X \iota$ ak pre *ľubovoľný* **ground term** c platí $\mathcal{I} \models_\top \iota[X|c]$,
> $\mathcal{I} \models_\bot \forall X \iota$ ak pre *niektorý* **ground term** c platí $\mathcal{I} \models_\bot \iota[X|c]$,
> inak $\mathcal{I} \models_{\Cup} \forall X \iota$.

Ostatné definície môžu ostať tak, ako boli popísané v predošlej kapitole.

1.3 Syntax rozšíreného datalogu

Zoberme si formálny zápis relatívne jednoduchej teórie s dotazom:

$$
\begin{aligned}
&[\forall X, Y\, (\text{parent}(X, Y) \Rightarrow \text{ancestor}(X, Y))] \wedge \\
&[\forall X, Y, Z\, ((\text{parent}(X, Z) \wedge \text{ancestor}(Z, Y)) \Rightarrow \text{ancestor}(X, Y))] \wedge \\
&[\forall X, Y\, (\text{ancestor}(X, Y))]
\end{aligned}
$$

Príklad 1.1

Tento zápis sa rýchlo stáva neprehľadným a dlhým. Preto Datalog používa skrátený zápis.

Klauzula

$$\forall X_1, \ldots, X_k((P_1 \wedge \cdots \wedge P_i \wedge \neg P_{i+1} \wedge \cdots \wedge \neg P_n) \Rightarrow Q),$$

kde $X_1, \ldots, X_k$ sú všetky premenné vyskytujúce sa v termoch použitých v predikátoch $P_1, \ldots, P_n$ a Q, sa v skrátenom zápise zapisuje ako

$$Q \leftarrow P_1, \ldots, P_i, \neg P_{i+1}, \ldots, \neg P_n.$$

Teórii s dotazom $K_1 \wedge K_2 \wedge \cdots \wedge K_n \wedge Q$ zodpovedá $D_1\, D_2 \ldots D_n\, ?Q$, kde D_i je skrátený zápis klauzuly K_i.

Skôr spomenutá teória je reprezentovaná skráteným zápisom:

$$\begin{aligned} &\text{ancestor}(X, Y) \leftarrow \text{parent}(X, Y). \\ &\text{ancestor}(X, Y) \leftarrow \text{parent}(X, Z), \text{ancestor}(Z, Y). \\ &\qquad\qquad ?\,\text{ancestor}(X, Y) \end{aligned}$$

Príklad 1.2

Poznámka 4. So znakom **t** , resp. **f** budeme označovať 0-árny predikát pre ktorý, pre každú interpretáciu $\mathcal{I}$, platí $\mathcal{I} \models_\top \mathbf{t}$, resp. $\mathcal{I} \models_\perp \mathbf{f}$.

Definícia 1.3.1. Klauzulu typu $p(t_1, \ldots, t_n) \leftarrow \mathbf{t}.$, kde p je n-árny predikát a $t_1, \ldots, t_n$ sú termy, nazývame **fakt**. Klasicky ich skrátene zapisujeme ako $p(t_1, \ldots, t_n)$. (zdôrazňujem, že bodka na konci je súčasťou skráteného zápisu). Keďže ale často potrebujeme vyjadriť pre jeden predikát viacero faktov, zvolíme nasledujúci zápis. Fakty

$$p(t_{1,1}, \ldots, t_{1,n}).$$

$$\vdots$$

$$p(t_{k,1}, \ldots, t_{k,n}).$$

zapíšem ako

$$p = \{(t_{1,1}, \ldots, t_{1,n}), \ldots, (t_{k,1}, \ldots, t_{k,n})\}.$$

(opäť zdôrazňujem, že bodka na konci je súčasťou zápisu).

V ASCII zápise datalogových programov a dotazov sa používajú alternatívne znaky, konkrétne:

- $\leftarrow \rightsquigarrow$ `:-`

- $\neg \rightsquigarrow$ `\+`
- Na označenie dotazov sa namiesto ? používa `?-`.

Doteraz spomínaná teória (rozšírená o fakty) by potom bola zapísaná ako:

```
ancestor = {(jozo, fero), (fero, sano)}.
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
?- ancestor(X, Y).
```

Príklad 1.3

1.4 Sémantika datalogu

Zoberme si veľmi jednoduchú teóriu:

```
p = {(1)}.
```

Pre túto teóriu existuje niekoľko modelov (dokonca ich je nekonečne veľa). Najjednoduchší z nich je $\mathcal{I} \models_\top \mathtt{p}(1)$ a pre všetky ostatné formuly je interpretácia nepravdivá. Ďalšími modelmi môžu byť napríklad:

- $\mathcal{I}' \models_\top \mathtt{p}(i), \forall i \in \mathbb{N}$ a pre všetky ostatné formuly je interpretácia nepravdivá,
- $\mathcal{I}'' \models_\top \mathtt{p}(i), \forall i \in \mathbb{R}$ a pre všetky ostatné formuly je interpretácia nepravdivá,
- $\mathcal{I}''' \models_\top \mathtt{p}(1), \mathcal{I}''' \models_\top \mathtt{p}(\text{Pes}), \mathcal{I}''' \models_\top p(47), \mathcal{I}''' \models_\top \mathtt{p}(\text{Štvorec})$ a pre všetky ostatné formuly je interpretácia nepravdivá.

Ako sémantiku našich datalogových programov budeme uvažovať *well-founded model*. Well-founded model je minimálny stabilný trojhodnotový model.

Trojhodnotový model znamená, že interpretácia priraďuje atomickým formulám hodnoty *true* $\top$, *false* $\bot$ a *unknown* $\Cup$.

Stabilný model znamená, že zostane rovnaký po tzv. *Gelfond-Lifschitz Transformation*. Aplikácia GLT funguje nasledovne: zoberieme aktuálnu interpretáciu a pomocou nej z programu odstránime tie pravidlá, ktorých negované podmienky sú podľa danej interpretácie pravdivé. Z ostatných pravidiel potom odstránime negácie. Tým dostaneme program bez negácie, pre ktorý vieme zostrojiť minimálny model. Ak sa tento

minimálny model zhoduje s interpretáciou, z ktorej sme vychádzali, hovoríme, že ide o stabilný model.

Zoberme si program

```
a.
b :- \+ c.
c :- \+ b.
d :- a, \+ c.
```

a kandidátnu interpretáciu $\mathcal{I}$, pre ktorú platí

$$\mathcal{I} \models_\top \mathtt{a}, \mathcal{I} \models_\top \mathtt{b}, \mathcal{I} \models_\bot \mathtt{c}, \mathcal{I} \models_\top \mathtt{d}.$$

Pri aplikácii GLT najprv odstránime tie pravidlá, ktorých negované podmienky sú podľa $\mathcal{I}$ pravdivé. Pravidlo `c :- \+ b.` sa preto odstráni, keďže `b` je v interpretácii $\mathcal{I}$ pravdivé. V ostatných pravidlách potom odstránime negácie, a dostaneme program

```
a.
b.
d :- a.
```

Minimálny model takto upraveného programu obsahuje práve atómy `a`, `b` a `d`. Keďže sa zhoduje s interpretáciou, z ktorej sme vychádzali, daná interpretácia je stabilný model pôvodného programu.

Minimálny model znamená, že spomedzi všetkých modelov vyberáme ten, ktorý obsahuje najmenej pravdivých atómov. Intuitívne teda nechceme odvodiť viac faktov, než je nutné. Ak existuje viac modelov, uprednostníme ten najmenší.

Príklad minimálneho modelu pre teóriu

```
p = {(1)}.
```

je modelom napríklad interpretácia, v ktorej platí iba $p(1)$. Modelom je aj interpretácia, v ktorej platí $p(1)$, $p(2)$ a $p(3)$. Minimálny model je však ten prvý, pretože obsahuje najmenej pravdivých atómov.

Well-founded model môžeme chápať ako niečo, čo je spoločné pre všetky stabilné modely. Atómy, ktoré sú pravdivé vo všetkých stabilných modeloch, označíme ako $\top$, atómy, ktoré sú nepravdivé vo všetkých stabilných modeloch, označíme ako $\bot$, a atómy, pri ktorých sa stabilné modely rozchádzajú, ponecháme ako $\Cup$.

V nasledujúcom príklade ukážeme, ako sa dá nájsť well-founded model teórie. Nejde o formálny dôkaz. Pre viacej o well-fouded model odporúčam prečítať [5] a [3].

Budeme postupovať iteratívne. Začneme interpretáciou, v ktorej sú všetky atómy nepravdivé, a budeme opakovane aplikovať GLT. Po každom kroku vezmeme minimálny model získaného programu a ten použijeme ako novú interpretáciu. Keď sa interpretácia prestane meniť, dostaneme výsledný model.

Príklad nájdenia well-founded modelu:

```
a.
b :- a.
p :- \+ q.
q :- \+ p.
u :- v.
```

Budeme uvažovať atómy `a`, `b`, `p`, `q`, `u` a `v`. Na začiatku zoberieme interpretáciu $\mathcal{I}_0$, v ktorej sú všetky tieto atómy nepravdivé.

Pri aplikácii GLT na $\mathcal{I}_0$ sa neodstráni žiadne pravidlo, pretože v $\mathcal{I}_0$ nie je pravdivý ani `p`, ani `q`. Po odstránení negácií dostaneme program

```
a.
b :- a.
p.
q.
u :- v.
```

Jeho minimálny model je interpretácia $\mathcal{I}_1$, v ktorej platí

$$\mathcal{I}_1 \models_\top \mathtt{a}, \mathcal{I}_1 \models_\top \mathtt{b}, \mathcal{I}_1 \models_\top \mathtt{p}, \mathcal{I}_1 \models_\top \mathtt{q}, \mathcal{I}_1 \models_\bot \mathtt{u}, \mathcal{I}_1 \models_\bot \mathtt{v}.$$

Teraz aplikujeme GLT na interpretáciu $\mathcal{I}_1$. Keďže v nej sú `p` aj `q` pravdivé, pravidlá `p :- \+ q.` aj `q :- \+ p.` sa odstránia. Po odstránení negácií dostaneme program

```
a.
b :- a.
u :- v.
```

Jeho minimálny model je interpretácia $\mathcal{I}_2$, v ktorej platí

$$\mathcal{I}_2 \models_\top \mathtt{a}, \mathcal{I}_2 \models_\top \mathtt{b}, \mathcal{I}_2 \models_\bot \mathtt{p}, \mathcal{I}_2 \models_\bot \mathtt{q}, \mathcal{I}_2 \models_\bot \mathtt{u}, \mathcal{I}_2 \models_\bot \mathtt{v}.$$

Ak teraz ešte raz aplikujeme GLT na interpretáciu $\mathcal{I}_2$, obe pravidlá s negáciou opäť zostanú, a preto znovu dostaneme program

```
a.
b :- a.
p.
q.
u :- v.
```

Jeho minimálny model je opäť $\mathcal{I}_1$. Dostali sme teda cyklus

$$\mathcal{I}_0 \to \mathcal{I}_1 \to \mathcal{I}_2 \to \mathcal{I}_3 \equiv \mathcal{I}_1 \to \mathcal{I}_2 \to \cdots$$

Vidíme, že atómy `a` a `b` sú pravdivé v každom kroku od $\mathcal{I}_1$ ďalej, a preto im vo well-founded modeli priradíme hodnotu $\top$. Atómy `u` a `v` nie sú pravdivé v žiadnom kroku, a preto im priradíme hodnotu $\bot$. Naopak, pri atómoch `p` a `q` sa interpretácie striedajú: raz sú pravdivé a raz nepravdivé. Tým pádom ich nevieme jednoznačne určiť, a vo well-founded modeli dostanú hodnotu $\Cup$.

Výsledný well-founded model tejto teórie je teda daný hodnotami

$$\mathtt{a} = \top, \qquad \mathtt{b} = \top, \qquad \mathtt{u} = \bot, \qquad \mathtt{v} = \bot, \qquad \mathtt{p} = \Cup, \qquad \mathtt{q} = \Cup.$$

Prehľadne to môžeme zapísať aj do tabuľky:

Atóm	$\mathcal{I}_0$	$\mathcal{I}_1$	$\mathcal{I}_2$	$\mathcal{I}_3$	$\mathcal{I}$
`a`	$\bot$	$\top$	$\top$	$\top$	$\top$
`b`	$\bot$	$\top$	$\top$	$\top$	$\top$
`u`	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
`v`	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
`p`	$\bot$	$\top$	$\bot$	$\top$	$\Cup$
`q`	$\bot$	$\top$	$\bot$	$\top$	$\Cup$

Kapitola 2

Relačná algebra

2.1 Definície

V tejto časti sa inšpirujeme relačnou algebrou z [1] a klasickou *učebnicovou* relačnou algebrou a definujeme našu vlastnú.

2.1.1 Učebnicová relačná algebra

Klasická relačná algebra pracuje s reláciami, ktoré majú väčšinou jednotlivé atribúty pomenované. Potom ich vieme jednoducho reprezentovať pomocou tabuliek.

$$R \in \textbf{Meno} \times \textbf{Výška}$$
$$R = \{(\text{Adam}, 175), (\text{Boris}, 183), (\text{Cecil}, 192)\}$$

R

Meno	**Výška**
Adam	175
Boris	183
Cecil	192

Príklad 2.1

Na manipuláciu s týmito reláciami existuje niekoľko operátorov:

Množinové operátory

zjednotenie množín $\cup$, rozdiel množín $-$, (prienik množín $\cap$, ale ten sa dá vyjadriť aj za pomoci rodielu $A \cap B = A - (A - B)$)

Pre relácie, na ktoré aplikujeme tieto operátory musí platiť, že majú identické domény atribútov. Pred tým, ako aplikujeme operátor na dve relácie, musíme usporiadať ich atribúty v jednotlivých záznamoch do rovnakého poradia.

Karteziánsky súčin $\times$

$R \times S$, kde R a S sú relácie. Výsledná relácia sa dá matematicky zapísať nasledovne:

$$R \times S = \{(a_1, \ldots, a_n, b_1, \ldots, b_m) \mid (a_1, \ldots, a_n) \in R, (b_1, \ldots, b_m) \in S)\}$$

R

X	**Y**
A	a
B	b
C	c

S

Y	**Z**
1	10
2	20
3	30

$R \times S$

X	**R.Y**	**S.Y**	**Z**
A	a	1	10
B	b	1	10
C	c	1	10
A	a	2	20
B	b	2	20
C	c	2	20
A	a	3	30
B	b	3	30
C	c	3	30

Príklad 2.2

Projekcia Π

$\Pi_{a_1,\ldots,a_n}(R)$, kde $a_1, \ldots, a_n$ sú názvy atribútov relácie R, vráti novú reláciu, ktorá obsahuje „iba stĺpce“ $a_1, \ldots, a_n$. Výsledok je stále relácia (čiže aj množina), takže neobsahuje duplicitné záznamy.

R

Meno	**Mesto**	**Zviera**
Adam	Bratislava	Pes
Boris	Bratislava	Mačka
Cecil	Košice	Pes
Dano	Lučenec	Zajac
Erik	Poprad	Kôň
Fero	Košice	Mačka

$\Pi_{\mathbf{Meno},\mathbf{Zviera}}(R)$

Meno	**Zviera**
Adam	Pes
Boris	Mačka
Cecil	Pes
Dano	Zajac
Erik	Kôň
Fero	Mačka

$\Pi_{\mathbf{Mesto}}(R)$

Mesto
Bratislava
Košice
Lučenec
Poprad

Príklad 2.3

Selekcia σ

$\sigma_C(R)$, kde C je podmienka, ktorá obsahuje atribúty relácie R. Výsledná relácia je podmnožina relácie R obsahujúca iba tie záznamy, ktoré spĺňajú podmienku C.

R

Meno	**Mesto**	**Zviera**
Adam	Bratislava	Pes
Boris	Bratislava	Mačka
Cecil	Košice	Pes
Dano	Lučenec	Zajac
Erik	Poprad	Kôň
Fero	Košice	Mačka

$\sigma_{\textbf{Mesto}=\text{Bratislava}\vee\textbf{Zviera}=\text{Mačka}}(R)$

Meno	**Mesto**	**Zviera**
Adam	Bratislava	Pes
Boris	Bratislava	Mačka
Fero	Košice	Mačka

Príklad 2.4

Prirodzený (natural) join $\bowtie$

$R \bowtie S$, kde R a S sú relácie. Podobne ako v karteziánskom súčine párujeme záznamy z dvoch relácií, ale iba tie záznamy z R a S, ktoré sa zhodujú na atribútoch spoločných pre R a S. V množinovom zápise:

$$
\begin{aligned}
R &= A_1 \times \cdots \times A_n \times C_1 \times \cdots \times C_k \\
S &= B_1 \times \cdots \times B_n \times C_1 \times \cdots \times C_k \\
R \times S &= \{(a_1, \ldots, a_n, c_1, \ldots, c_k, b_1, \ldots, b_m) \mid \quad (a_1, \ldots, a_n, c_1, \ldots, c_k) \in R, \\
&\qquad (b_1, \ldots, b_m, c'_1, \ldots, c'_k) \in S, \\
&\qquad c_1 = c'_1, \ldots, c_k = c'_k\}
\end{aligned}
$$

R

Meno	**Mesto**
Adam	Bratislava
Boris	Bratislava
Cecil	Košice
Dano	Lučenec
Erik	Poprad
Fero	Košice

S

Mesto	**Rozloha**
Bratislava	368
Košice	242
Lučenec	47
Poprad	63

$R \bowtie S$

Meno	**Mesto**	**Rozloha**
Adam	Bratislava	368
Boris	Bratislava	368
Cecil	Košice	242
Dano	Lučenec	47
Erik	Poprad	63
Fero	Košice	242

Príklad 2.5

Antijoin $\triangleright$

$R \triangleright S$, kde R a S sú relácie. Vo výslednej relácii sú tie záznamy z R, pre ktoré neexistuje záznam v S, s ktorým by sa zhodovali na spoločných atribútoch. V množinovom zápise:

$$R = A_1 \times \cdots \times A_n \times C_1 \times \cdots \times C_k$$
$$S = B_1 \times \cdots \times B_n \times C_1 \times \cdots \times C_k$$
$$R \times S = \{(a_1, \ldots, a_n, c_1, \ldots, c_k) \in R \,|\, \forall (b_1, \ldots, b_m, c'_1, \ldots, c'_k) \in S : c_1 \neq c'_1, \ldots, c_k \neq c'_k\}$$

Z

Meno	**Mesto**
Adam	Bratislava
Boris	Bratislava
Cecil	Košice
Dano	Lučenec
Erik	Poprad
Fero	Košice

V

Mesto
Bratislava
Lučenec

$Z \triangleright V$

Meno	**Mesto**
Cecil	Košice
Erik	Poprad
Fero	Košice

Príklad 2.6

Theta-join $\bowtie_C$

$R \bowtie_C S$, kde R a S sú relácie a C je podmienka, v ktorej sa vyskytujú atribúty z R a S. Natural join nám povolil spojiť dva záznamy iba ak ich spoločné atribúty boli rovnaké. Theta-join popáruje každú dvojicu záznamov z R a S, ktoré spĺňajú podmienku C.

M

Mesto	**Roz**
Bratislava	368
Košice	242
Lučenec	47
Poprad	63

R

Od	**Do**	**Veľ**
0	100	Malé
101	300	Stredné
301	1000	Veľké

$M \bowtie_{R.\mathbf{Od} < M.\mathbf{Roz} \wedge M.\mathbf{Roz} < R.\mathbf{Od}} R$

Mesto	**Roz**	**Od**	**Do**	**Veľ**
Bratislava	368	301	1000	Veľké
Košice	242	101	300	Stredné
Lučenec	47	0	100	Malé
Poprad	63	0	100	Malé

Príklad 2.7

Premenovanie ρ

$\rho_{A/B}(R)$, kde R je relácia, A je názov atribútu z relácie R a B je nový názov atribútu. Výsledkom bude relácia s rovnakými záznamami, iba atribút A sa premenuje na B.

R

Meno	**Mesto**	**Zviera**
Adam	Bratislava	Pes
Boris	Bratislava	Mačka
Cecil	Košice	Pes
Dano	Lučenec	Zajac
Erik	Poprad	Kôň
Fero	Košice	Mačka

$\rho_{\mathbf{Mesto/Obec}}(R)$

Meno	**Obec**	**Zviera**
Adam	Bratislava	Pes
Boris	Bratislava	Mačka
Cecil	Košice	Pes
Dano	Lučenec	Zajac
Erik	Poprad	Kôň
Fero	Košice	Mačka

Príklad 2.8

Iné join-y

V niektorých knihách sa spomínajú aj iné variácie join-ov ako *left outer join*, *right outer join* alebo *full outer join*. My sa s nimi nebudeme zaoberať, keďže sa dajú vyskladať z predchádzajúcich operátorov. Pre ich správne fungovanie treba ale zaviesť špeciálnu konštantu, ktorá sa v literatúre označuje ako ω, ε, `null` alebo `none`.

2.1.2 Naša relačná algebra

Pre jednoduchší preklad datalogu do relačnej algebry sme si zvolili nasledujúcu algebru so štyrmi operátormi (inšpirovaná algebrou z [1]). Tiež pracuje s reláciami, ale nepotrebuje vopred pomenované atribúty. Vždy, keď potrebuje pracovať s „názvami stĺpcov“, sú explicitne vymenované.

UNION$([R_1, \ldots, R_n])$, kde $R_1, \ldots, R_n$ sú relácie s rovnakým počtom atribútov. Výsledná relácia bude obsahovať všetky záznamy z každej relácie R_1 až R_n (až na duplikáty). Formálne **UNION**$([R_1, \ldots, R_n]) = \bigcup_{i=1}^{n} R_i$.

A	
11	a
12	b
13	c

B	
21	a
22	b
23	c

C	
31	a
32	b
33	c

UNION$([A, B, C])$	
11	a
12	b
13	c
21	a
22	b
23	c
31	a
32	b
33	c

Príklad 2.9

JOIN$([R_1, \ldots, R_n], [[T_{1,1}, \ldots, T_{1,a_1}], \ldots, [T_{n,1}, T_{n,a_n}]], [T_1, \ldots, T_k])$, kde R_i je relácia s aritou a_i, $T_{i,j}$ sú *vstupné* termy a T_l sú *výstupné* termy.

Výsledná relácia bude nasledujúca. Nech $P_1, \ldots, P_p$ sú všetky premenné vyskytujúce sa v termoch $T_{1,1}, \ldots, T_{1,a_1}, T_{2,1}, \ldots, T_{n,a_n}$. Nech množina M obsahuje všetky $(c_1, \ldots, c_p)$ také, pre ktoré platí

$$(T_{i,1}[P_1|c_1, \ldots, P_p|c_p], \ldots, T_{i,a_i}[P_1|c_1, \ldots, P_p|c_p]) \in R_i, \forall i \in \{1, \ldots, n\}.$$

Potom vo výslednej relácii R budú záznamy

$$R = \{(T_1[P_1|k_1, \ldots, P_p|k_p], \ldots, T_k[P_1|k_1, \ldots, P_p|K_p]) \mid (k_1, \ldots, k_p) \in M\}.$$

G	
1	2
2	3
3	4
3	5

JOIN$([G, G], [[X, Z], [Z, Y]], [X, Y])$	
1	3
2	4
2	5

Príklad 2.10

ANTIJOIN$([R_1, \ldots, R_n], [[T_{1,1}, \ldots, T_{1,a_1}], \ldots, [T_{n,1}, T_{n,a_n}]], [T_1, \ldots, T_k])$, kde R_i je relácia s aritou a_i, $T_{i,j}$ sú *vstupné* termy a T_l sú *výstupné* termy.

Výsledná relácia bude nasledujúca. Nech $P_1, \ldots, P_p$ sú všetky premenné vyskytujúce sa v termoch $T_{1,1}, \ldots, T_{1,a_1}, T_{2,1}, \ldots, T_{n,a_n}$. Nech množina M obsahuje všetky $(c_1, \ldots, c_p)$ také, pre ktoré platí

$$(T_{1,1}[P_1|c_1, \ldots, P_p|c_p], \ldots, T_{1,a_1}[P_1|c_1, \ldots, P_p|c_p]) \in R_1$$

a zároveň pre *žiadne* $i \in \{2, \ldots, n\}$ *neplatí*

$$(T_{i,1}[P_1|c_1, \ldots, P_p|c_p], \ldots, T_{i,a_i}[P_1|c_1, \ldots, P_p|c_p]) \in R_i.$$

Potom vo výslednej relácii R budú záznamy

$$R = \{(T_1[P_1|k_1, \ldots, P_p|k_p], \ldots, T_k[P_1|k_1, \ldots, P_p|K_p]) \mid (k_1, \ldots, k_p) \in M\}.$$

Nech f je funkčný symbol, ktorý pre ľubovoľný reťazec vráti jeho prvý znak.

U	
Adam	Bratislava
Boris	Bratislava
Cecil	Košice
Dano	Lučenec
Erik	Poprad
Fero	Košice

P
B
K

ANTIJOIN$([U, P], [[M, B], [f(B)]], [M])$
Adam
Dano
Erik
Fero

Príklad 2.11

REC(N, R), kde N je názov relácie a R je relácia. Tento operátor definuje reláciu s názvom N a dovoľuje použiť v relácii R odkaz na ňu.

$$\mathbf{REC}(t, \mathbf{UNION}([\$s, \mathbf{JOIN}([t, t], [[X, Z], [Z, Y]], [X, Y])]))$$

Relácia t je tranzitívny uzáver vstavanej relácie s.

Príklad 2.12

2.2 Inklúzia medzi učebnicovou a našou relačnou algebrou

V nasledujúcej kapitole sa pokúsime ukázať, že naša relačná algebra má aspoň takú výpočtovú silu ako učebnicová relačná algebra. To znamená, že každý výraz v učebnicovej

relačnej algebre môžeme vyjadriť pomocou výrazov v našej relačnej algebre. Dôkazy nebudú úplne formálne správne, ale mali by byť dostatočne jasné a zrozumiteľné, aby bolo zrejmé, ako by sa mal ľubovoľný program v učebnicovej relačnej algebre prepísať do našej algebry. Pre viac informácií o našej relačnej algebre odporúčame pozrieť [1] a [2].

Množinové operátory

Zjednotenie množín (relacií) $R \cup S$ vieme prepísať ako **UNION**$([R, S])$. Rozdiel množín $R - S$ prepíšeme ako

$$\textbf{ANTIJOIN}([R, S], [[A_1, A_2, \ldots, A_n], [A_1, A_2, \ldots, A_n]], [A_1, A_2, \ldots, A_n]).$$

Keďže prienik vieme vyjadriť pomocou rozdielu, budeme ho vedieť zapísať pomocou **ANTIJOIN**.

Karteziánsky súčin $\times$

Karteziánsky súčin $R \times S$ vieme prepísať pomocou **JOIN**. Jednoducho vymenujeme všetky atribúty z R a S ako vstupné termy a výstupné termy.

$$R \times S \approx \textbf{JOIN}([R, S], [[A_1, A_2, \ldots, A_n], [B_1, B_2, \ldots, B_m]], [A_1, A_2, \ldots, A_n, B_1, B_2, \ldots, B_m])$$

Projekcia Π

Projekcia je už zakomponovaná v **JOIN** a **ANTIJOIN**. Ak ale chceme vyjadriť iba projekciu, môžeme použiť **JOIN** s pomenovanými vstupnými termami a vymenovanými výstupnými termami, ktoré sú podmnožinou vstupných termov. Napríklad $\Pi_{A_1, A_3}(R)$ prepíšeme ako

$$\textbf{JOIN}([R], [[A_1, A_2, A_3, \ldots, A_n]], [A_1, A_3]).$$

Treba si ale dávať pozor, že v našej algebre záleží na poradí atribútov a v učebnicovej algebre záleží iba na ich názvoch. Preto je rozdiel medzi

$$\textbf{JOIN}([R], [[A_1, A_2, \mathbf{A_3}]], [A_1, \mathbf{A_3}]),$$

$$\textbf{JOIN}([R], [[A_1, \mathbf{A_3}, A_2]], [A_1, \mathbf{A_3}]) \text{ a}$$

$$\textbf{JOIN}([R], [[A_1, A_2, \mathbf{A_3}]], [\mathbf{A_3}, A_1]).$$

Selekcia σ

Pre selekciu využijeme veľkú výpočtovú silu funkčných symbolov. Zoberme si selekciu $\sigma_C(R)$. Skonštruujeme funkčný symbol f_C tak, že $f_C(p_1, p_2, \ldots, p_n) = \mathbf{t}$ práve vtedy, keď podmienka C platí pre záznam $(p_1, p_2, \ldots, p_n)$. Pre všetky ostatné vstupy bude f_C vracať $\mathbf{f}$. Potom môžeme selekciu $\sigma_C(R)$ prepísať ako

$$\mathbf{JOIN}([R, \$true], [[p_1, p_2, \ldots, p_n], [f_C(p_1, p_2, \ldots, p_n)]], [p_1, p_2, \ldots, p_n, \mathbf{t}]).$$

Výsledná relácia má ale o jeden atribút viac, preto ju treba ešte *obaliť* projekciou (joinom), ktorá nám tento atribút odstráni.

Prirodzený (natural) join $\bowtie$

Opäť si musíme dávať pozor na poradie atribútov. Preklad $R \bowtie S$ bude ale potom priamočiary. Ukážeme na príklade, keď relácie R a S majú spoločné atribúty $C_1, C_2, \ldots, C_k$ *na konci.*

$$\begin{aligned} R \bowtie S \approx \mathbf{JOIN}([R, S], & \\ & [[A_1, \ldots, A_n, C_1, \ldots, C_k], [B_1, \ldots, B_m, C_1, \ldots, C_k]], \\ & [A_1, \ldots, A_n, C_1, \ldots, C_k, B_1, \ldots, B_m]) \end{aligned}$$

Antijoin $\triangleright$

Antijoin $R \triangleright S$ prepíšeme v podstate rovnako ako natural join. Znovu predpokladáme, že spoločné atribúty $C_1, C_2, \ldots, C_k$ sú *na konci.*

$$\begin{aligned} R \triangleright S \approx \mathbf{ANTIJOIN}([R, S], & \\ & [[A_1, \ldots, A_n, C_1, \ldots, C_k], [B_1, \ldots, B_m, C_1, \ldots, C_k]], \\ & [A_1, \ldots, A_n, C_1, \ldots, C_k]) \end{aligned}$$

Theta-join $\bowtie_C$

Ako pri selekcii, aj tu využijeme funkčné symboly. Zoberme si theta-join $R \bowtie_C S$. Skonštruujeme funkčný symbol f_C tak, že $f_C(p_1, p_2, \ldots, p_n, q_1, q_2, \ldots, q_m) = \mathbf{t}$ práve vtedy, keď podmienka C platí pre záznam $(p_1, p_2, \ldots, p_n)$ z relácie R a záznam $(q_1, q_2, \ldots, q_m)$ z relácie S. Pre všetky ostatné vstupy bude f_C vracať $\mathbf{f}$. Potom dokážeme theta-join $R \bowtie_C S$ prepísať ako

$$\begin{aligned} \mathbf{JOIN}([R, S, \$true], & \\ & [[A_1, \ldots, A_n], [B_1, \ldots, B_m], [f_C(A_1, \ldots, A_n, B_1, \ldots, B_m)]], \\ & [A_1, \ldots, A_n, B_1, \ldots, B_m, \mathbf{t}]) \end{aligned}$$

Výsledná relácia má ale znovu o jeden atribút viac, preto ju znovu treba *obaliť* projekciou (joinom), pomocou ktorej tento atribút odstránime.

Premenovanie ρ

Keďže v **JOIN** a **ANTIJOIN** zakaždým explicitne vymenujeme všetky vstupné a výstupné termy, premenovanie ρ nepredstavuje žiaden problém.

Kapitola 3

Kompilátor

3.1 ANTLR

Náš kompilátor používa generátor *parserov* **AN**other **T**ool for **L**anguage **R**ecognition. V tejto časti si priblížime ako *ANTLR* funguje a čo motivovalo zmeny v relačnej algebre.

3.2 Gramatika datalogu pre ANTLR

Program ANTLR potrebuje na vygenerovanie parseru gramatiku parsovaného jazyka.

3.3 Algoritmus prekladu

Po vygenerovaní stromu odvodenia prichádza na rad preklad do relačnej algebry.

Záver

Literatúra

[1] Diana Biriukova. Compiler of relational algebra expressions, 2025.

[2] Tomáš Magát. Todo, 2026.

[3] Tomáš Plachetka and Ján Šturc. Semantics of queries. Prednáškové slajdy ku kurzu Databases, Fakulta matematiky, fyziky a informatiky Univerzity Komenského v Bratislave, February 2026. Summer 2025–2026. Dostupné na: `http://www.dcs.fmph.uniba.sk/~plachetk/TEACHING/RLDB2026/index.html`.

[4] Soufflé Language Team. Soufflé: Program translation. `https://souffle-lang.github.io/translate`, 2024. [Online; navštívené 7. januára 2026].

[5] Allen Van Gelder, Kenneth A. Ross, and John S. Schlipf. The well-founded semantics for general logic programs. *Journal of the ACM*, 38(3):620–650, July 1991.